

TinyML-Based Indoor Air Quality Prediction and Ventilation Control System

S.M. Sifatul Islam (011 222 020) Kazi Md. Juhaer Akhtab (011 222 112) Junayed Hossain (011 222 056)

Md Shamsul Huda (011 211 120) Nirob Chandra Banik (011 222 265) Shadik Hasan (011 222 026)

Abstract—Indoor air pollution is a growing concern as poor air quality can directly affect human health and comfort in indoor environments. Continuous monitoring and timely control of ventilation systems are required to maintain a healthy indoor atmosphere.

This project presents an IoT-based smart air quality monitoring and ventilation control system using Arduino, Raspberry Pi, and environmental sensors. Sensor data such as temperature, humidity, and air quality are collected using Arduino and transmitted to a Raspberry Pi, where a machine learning model and rule-based logic classify the air quality as good, moderate, or poor. Based on the classification, the system automatically controls a cooling fan and a servo driven window mechanism and displays real-time information on an OLED display.

Experimental results show that the system responds effectively to changing air quality conditions with minimal delay, providing reliable automation and efficient ventilation control for indoor environments.

Index Terms—TinyML, Indoor Air Quality, Raspberry Pi, Arduino, Embedded Systems, IoT, Smart Ventilation

I. INTRODUCTION

Indoor air pollution is a significant concern in residential, educational, and industrial environments due to the accumulation of harmful gases, temperature variation, and humidity imbalance. Prolonged exposure to poor indoor air quality can lead to respiratory issues, discomfort, and reduced productivity. Traditional air quality monitoring systems mostly rely on fixed threshold-based techniques and manual intervention, which are not adaptive to varying environmental conditions. The motivation behind this project is to design an intelligent, low-cost, and autonomous system that can continuously monitor indoor air quality and take corrective actions without human intervention. With recent advancements in TinyML, it has become possible to deploy machine learning models directly on edge devices such as Raspberry Pi, enabling real-time prediction and decision-making. Therefore, this project focuses on integrating TinyML with embedded systems to automatically control ventilation mechanisms based on predicted air quality levels. Recently advances in Tiny Machine Learning (TinyML) and edge intelligence have enabled the deployment of lightweight machine learning models on resource-constrained devices for real-time environmental monitoring. Datta et al. [1] showed that it is possible to predict the Air Quality Index (AQI) directly on TinyML-based devices. They

used simple weather-related data, such as temperature and time information, along with lightweight machine learning models. Their study demonstrates that air quality prediction can be done on the device itself without relying on cloud servers, while still achieving reasonable accuracy and fast response times. This approach is especially useful for personal and small-scale air quality monitoring systems. Kudrin-Gusten et al. [2] proposed a hazardous gas detection system integrated into intelligent personal protective equipment (PPE). Their TinyML-based approach, implemented on an ESP32-C6 microcontroller, achieved faster response times and higher accuracy than traditional threshold-based methods, proving its robustness in dynamic environments. In the healthcare domain, Omprakash et al. [3] introduced an edge-intelligent, IoT-enabled ventilation system that employed TinyML models to control oxygen delivery based on physiological and environmental sensor data. Their system demonstrated high prediction accuracy and effective anomaly detection under resource constraints. Datta et al. [4] presented a TinyML-based air quality monitoring framework with integrated sensor fault detection. By combining direct AQI estimation with secondary TinyML-based sensing, the system enabled continuous and reliable long-term monitoring. Despite these advancements, existing studies largely address prediction, detection, or fault tolerance in isolation, with limited focus on integrating real-time TinyML inference with actuator-based environmental control under strict hardware constraints.

The workflow and system architecture of our project differ from those of existing works. In our design, a microcontroller is responsible for sensor data acquisition, while a separate microprocessor is used to execute the TinyML model. Based on the model's predictions, control commands are transmitted to an Arduino, which actuates a servo motor to regulate window opening. This separation of sensing, inference, and actuation introduces a distinct workflow and functional architecture compared to existing approaches.

II. PROPOSED METHOD

A. Block diagram of the overall system

The overall system consists of three main hardware components: the Arduino, the Raspberry Pi, and the peripheral devices including sensors, a DC 5V Exhaust fan, a servo motor, and an OLED display. The DHT11 temperature and humidity sensor and the MQ-135 air quality sensor are connected to the

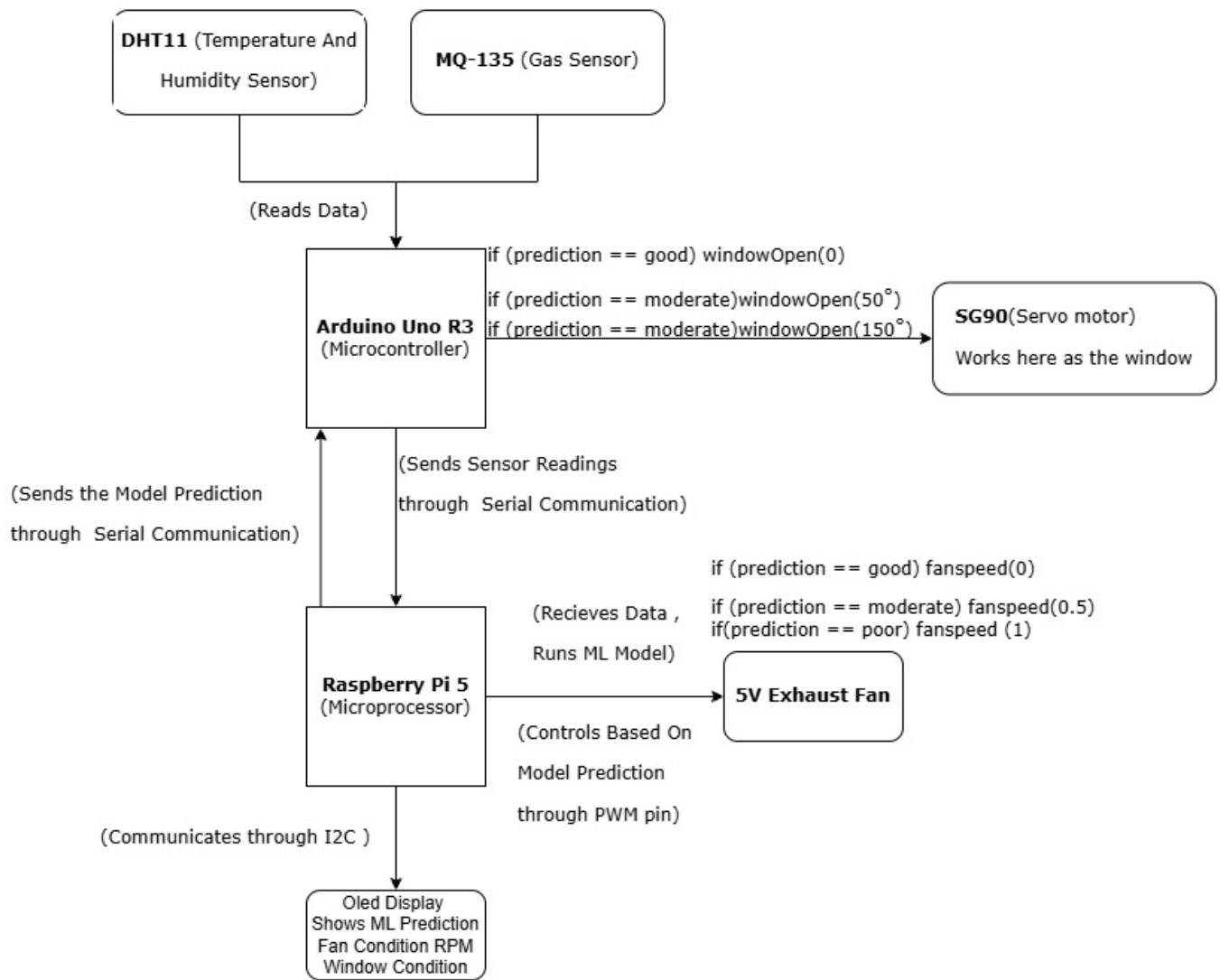


Fig. 1. Block Diagram of the System

Arduino, which continuously reads their data. Fig.1 represents the Block Diagram of the overall system

The Arduino then sends these sensor readings to the Raspberry Pi over a Serial communication. The Raspberry Pi acts as the central processing unit, receiving the data, running the ML model to classify air quality as “GOOD,” “MODERATE,” or “POOR,” and making real-time control decisions. Based on the predicted air quality, the Raspberry Pi directly controls a 5V fan through a PWM signal on GPIO, adjusting the fan speed accordingly. Simultaneously, the Raspberry Pi sends commands back to the Arduino over Serial Communication, instructing it to adjust the servo motor to control the window position, with positions corresponding to “CLOSED,” “PARTIAL,” or “FULL OPEN.” Additionally, the Raspberry Pi updates an OLED display via I²C communication, showing temperature, humidity, air quality readings, fan speed in RPM, window position, and the air quality

classification. This configuration ensures a continuous loop of sensing, processing, actuation, and display, where the Arduino handles sensor interfacing and servo control, the Raspberry Pi handles sensor computation and fan control, and the OLED provides a real-time visual feedback of the system’s status.

III. IMPLEMENTED HARDWARE SYSTEM

A. List of the hardwares and software Environment

1) **Hardware Components:**

- Arduino Uno R3 (microcontroller)
- Raspberry Pi 5 (microprocessor)
- DHT11 temperature and humidity sensor
- MQ135 air quality (gas) sensor
- SG90 servo motor
- DC cooling fan 5V 40 x 40 x 7mm
- OLED display (SSD1306)
- NPN Transistor (BC547)

- Resistor (1K Ohm)
- Wires

2) *Software Environment:*

- Arduino IDE
- Raspberry Pi OS
- Python 3
- Edge Impulse Studio
- TinyML Frameworks (Edge Impulse SDK)
- Geany

B. *Figures of the implemented hardware system*

This section shows the figures of the implemented hardware system. Each figure explains a different step of how the system works, starting from collecting environmental data, then processing and making decisions, followed by controlling the actuators, and finally showing the complete integrated system. The figures are labeled to clearly show the main components and how data flows between them, which makes easier for everyone to understand the overall system operation. **Fig. 2** shows the **sensors (DHT11 and MQ-135)** measuring environmental data. The temperature, humidity, and air quality levels are captured in real-time, which forms the input for the control system.

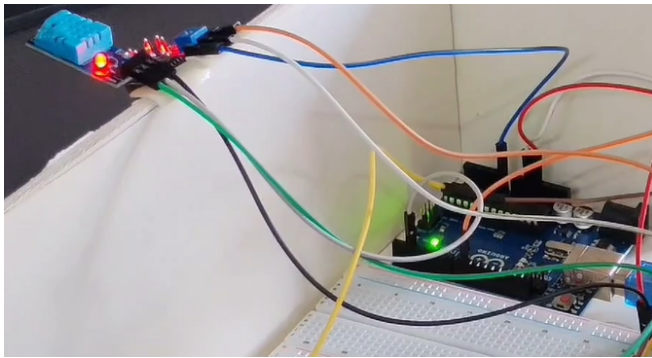


Fig. 2. Sensor data readings.

Fig.3 illustrates that Arduino sending the sensor readings to the Raspberry Pi 5 via a Serial connection. This figure highlights the data transfer pathway from the microcontroller to the processing unit.

Fig. 4 demonstrates that Raspberry Pi running the ML model to predict air quality. Based on the prediction, the Raspberry Pi adjusts the fan speed using PWM output to control airflow.

Fig. 5 shows that Raspberry Pi sending predictions back to the Arduino, which then drives the **servo motor**. The servo motor adjusts the window position, opening or closing it according to the air quality classification.

Fig. 6 presents the OLED display showing all real-time information, including temperature, humidity, air quality reading, fan RPM, window position, and air quality prediction (GOOD, MODERATE, POOR).

Fig. 7 provides an overall view of the entire system, showing the integration of all components such as sensors, Arduino,

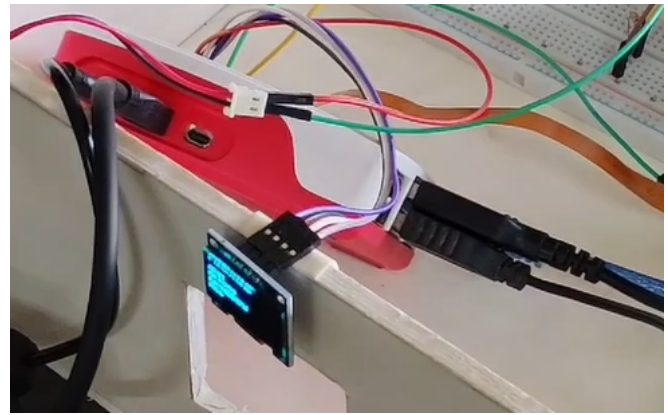


Fig. 3. Data transmission to Raspberry Pi 5.



Fig. 4. Air quality prediction and fan control.

Raspberry Pi, fan, servo motor, and OLED display and their connections, giving a clear perspective of the implemented hardware setup.

The working demonstration of the proposed system is shown in the project presentation video [5].

IV. RESULTS

This section presents the experimental results obtained from the implemented smart air quality monitoring and control system. The results are discussed step by step to clearly demonstrate how sensor data is collected, processed by the machine learning model, and translated into physical actions through actuators such as the fan and servo motor. Initially, the environmental sensors (DHT11 and MQ135) continuously monitor temperature, humidity, and air quality. Fig. 2 shows sample sensor readings captured under different environmental conditions. These readings are transmitted from the Arduino to the Raspberry Pi 5 via serial communication, as illustrated in Fig. 3.

Once the data reaches the Raspberry Pi, the trained Edge Impulse machine learning model processes the input values and classifies the air quality into one of three categories: *Good*, *Moderate*, or *Poor*. Based on this prediction, the Raspberry Pi generates control decisions for both ventilation speed and

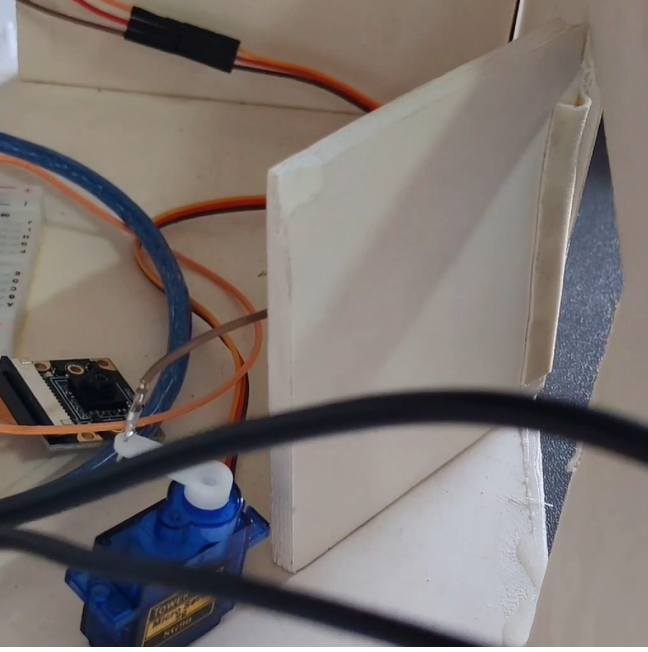


Fig. 5. Servo motor control via Arduino.



Fig. 6. OLED display output

window position. Figure 4 demonstrates the ML inference process and corresponding fan actuation.

For a test case with temperature 40 °C, humidity 10%, and air quality sensor value 451, the model predicted a *Poor* condition. As shown in Fig. 6, the fan operated at full speed (3000 RPM) to maximize air circulation. Simultaneously, a control signal was sent back to the Arduino, which rotated the SG90 servo motor to 150°, fully opening the window to allow maximum ventilation.

In a second scenario, where the temperature was 30 °C, humidity was 61%, and air quality value was 330, the model classified the condition as *Moderate*. In this case, the fan ran at a medium speed of approximately 1500 RPM, and the servo

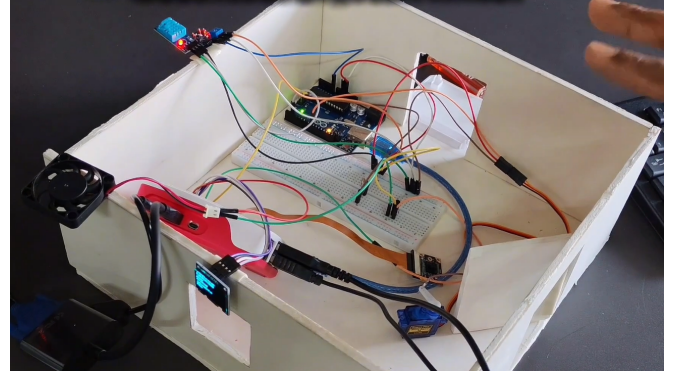


Fig. 7. Overall system implementation.

motor rotated to 50°, resulting in a partially opened window.

For favorable conditions with temperature 27 °C, humidity 70%, and air quality sensor value 120, the model predicted a *Good* condition. Here the fan remained switched off, and the servo motor position was set to 0°, keeping the window fully closed. This confirms that the system avoids unnecessary actuation and reduces energy consumption when air quality is acceptable.

Additionally, all real-time parameters including sensor values, predicted air quality class, fan speed in RPM, and window status are displayed on the OLED screen, as depicted in Fig. 6. Figure 7 presents the complete integrated hardware setup of the system.

From these results, it is evident that the proposed system responds accurately and reliably to varying environmental conditions. The machine learning predictions are correctly mapped to actuator control decisions, ensuring efficient ventilation, improved indoor air quality, and optimized energy usage.

V. DISCUSSION

The proposed Smart Air Quality Monitoring and Automated Ventilation System requires deep and diverse knowledge across multiple engineering and computing domains. The project integrates embedded systems (Arduino-based sensor interfacing with DHT11 and MQ-135) (**K3, K4**), computer architecture and hardware interfacing (GPIO, PWM control, transistor-based fan driving, servo motor actuation) (**K5**), and Linux-based single-board computing using Raspberry Pi (**K4**). Also, the system involves machine learning concepts such as feature extraction, real-time inference, and classification using an Edge Impulse trained model (**K6, K8**). Knowledge of serial communication protocols (UART) (**K3**), I2C communication for OLED display (**K3**), and software integration using Python is also essential. Furthermore, understanding of control systems, environmental monitoring, and energy-efficient automation principles strengthens the depth of knowledge involved.

Based on this multidisciplinary knowledge requirement, the project clearly satisfies **P1 (Depth of Knowledge)**. Table I presents the knowledge profiles corresponding to these attributes.

TABLE I
KNOWLEDGE PROFILES

K1	K2	K3	K4	K5	K6	K7	K8
		✓	✓	✓	✓		✓

The project addresses multiple conflicting requirements, particularly between system performance, safety, and cost. For example, controlling a 5V DC fan requires sufficient power and speed control, but directly powering it from the Raspberry Pi GPIO could damage the hardware. This conflict was resolved by introducing a transistor-based switching circuit, ensuring both safety and functionality. Another conflict arises between real-time processing accuracy and computational limitations of edge devices. While higher model complexity improves prediction accuracy, it also increases latency and power consumption. Therefore, a balance was maintained by deploying a lightweight ML model suitable for edge inference. These trade-offs confirm the presence of conflicting requirements, satisfying **(P2 Range of Conflicting Requirements)**. The project consists of several interdependent hardware and software modules, including the Arduino based sensor unit, Raspberry Pi processing unit, machine learning inference engine, fan control circuit, servo motor mechanism, and OLED display. Changes in one module such as sensor readings or ML model output—directly affect the whole system behavior like fan control, servo motor rotation etc. The modules are tightly coordinated through serial communication, exhibiting strong hardware and software interdependence despite being decoupled across devices (Arduino and Raspberry Pi) which satisfies **(P7 Interdependence)**.

TABLE II
COMPLEX ENGINEERING PROBLEM MAPPING

P1	P2	P3	P4	P5	P6	P7
✓	✓					✓

VI. CONCLUSION

This project successfully demonstrates a TinyML-based smart indoor air quality monitoring and automatic ventilation system. The system efficiently classifies air quality in real time and autonomously controls ventilation mechanisms. Future enhancements include integrating advanced air quality sensors and improving the machine learning model accuracy for more reliable classification. Wireless communication can be deployed to enable remote monitoring and control through a mobile application. Also, to reduce energy consumption, a PIR sensor can be integrated so that the system remains inactive when no occupants are detected inside the environment.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our mentor **Mr. Md. Shafqat Talukder** for his continuous guidance, encouragement, and valuable technical insights throughout the

development of this project. His support and constructive feedback played an important role in shaping the system design and implementation. He helped us overcome technical issues and guided us in designing proper data flow and system flow throughout the project. We also gratefully acknowledge the support of the department laboratory, particularly **Room No. 506**, where essential components such as transistors, resistors, and sensors were provided and tested during the project development.

REFERENCES

- [1] A. Datta, A. Pal, R. Marandi, N. Chattaraj, S. Nandi, and S. Saha, "Efficient air quality index prediction on resource-constrained devices using tinyml: Design, implementation, and evaluation," in *Proceedings of the 25th International Conference on Distributed Computing and Networking*, 2024, pp. 304–309.
- [2] C. Kudrin-Gusten, M. Kuhl, V. Barth, and H. Yu, "Tinyml-based hazardous gas detection for intelligent personal protective equipment," in *2025 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRINE)*, 2025, pp. 25–30.
- [3] M. Omprakash, S. Saisukirtha, S. Shane Steffy, and S. Dilip Kumar, "IoT-enabled smart ventilation system with edge-based oxygen regulation and embedded clinical training," in *2025 9th International Conference on Inventive Systems and Control (ICISC)*. IEEE, 2025, pp. 990–996.
- [4] A. Datta, J. Chakraborty, P. Majhi, A. Mondal, S. Roy, S. Nandi, and S. Saha, "Pollusensecheck: Cost effective tinyml-based air quality monitoring system with in-built sensor fault detection," in *2025 17th International Conference on COMMunication Systems and NETworks (COMSNETS)*. IEEE, 2025, pp. 623–631.
- [5] S. M. S. Islam, K. M. J. Akhtab, J. Hossain, M. S. Huda, N. C. Banik, and S. H. Bappy, "Tinyml-based indoor air quality prediction and ventilation control system – project presentation," Video presentation, 2025. [Online]. Available: https://drive.google.com/file/d/1MROwH9SBpOD_QHrbicEzA3MD9c_ZuSn/view